Part 1

made use of comm_test .c files
timing _comm …c file

Running code as is

```
fasetid2@cheetah:~$ mpicc PHY1090/week4/comm_test_mpi.c -o bin/comm_test_mpi
fasetid2@cheetah:~$ mpirun -np 4 bin/comm_test_mpi
No protocol specified
Hello, I am 1 of 4. Sent 10 to Rank 0
Hello, I am 0 of 4. Received 10 from Rank 1
Hello, I am 2 of 4. Sent 20 to Rank 0
Hello, I am 3 of 4. Sent 30 to Rank 0
Hello, I am 0 of 4. Received 20 from Rank 2
Hello, I am 0 of 4. Received 30 from Rank 3
```

Process where varied form 4,,2,8,16 and what was observed is all non-root processes sent a value equal to rank x 10 to rank 0, and rank 0 then received and printed those values. The outputs showed that the order in which the "Sent" messages appeared was not fixed and varied between runs. This is expected in MPI, because multiple processes are executing concurrently and their output is not synchronised. The root process still successfully received the correct values from each rank. the experiment demonstrates that parallel programs may produce output in different orders even when the communication itself is correct

Code was functionalized and still works

Experimenting with send types

MPI_Ssend()

```
fasetid2@cheetah:~$ mpirun -np 8  bin/my_comm_test_mpi1
No protocol specified
Hello, I am 0 of 8. Received 10 from Rank 1
Hello, I am 0 of 8. Received 20 from Rank 2
Hello, I am 0 of 8. Received 30 from Rank 3
Hello, I am 0 of 8. Received 40 from Rank 4
Hello, I am 0 of 8. Received 50 from Rank 5
Hello, I am 0 of 8. Received 60 from Rank 6
Hello, I am 0 of 8. Received 70 from Rank 7
Hello, I am 1 of 8. Sent 10 to Rank 0
Hello, I am 2 of 8. Sent 20 to Rank 0
Hello, I am 3 of 8. Sent 30 to Rank 0
Hello, I am 4 of 8. Sent 40 to Rank 0
Hello, I am 5 of 8. Sent 50 to Rank 0
Hello, I am 6 of 8. Sent 60 to Rank 0
Hello, I am 7 of 8. Sent 70 to Rank 0
```

The process are ordered and not random . behaved reliably and produced the expected output. Since it is a synchronous send, it waits until the matching receive is posted, which can make behaviour more predictable

MPI_Bsend ()

```
fasetid2@cheetah:~$ mpirun -np 10 bin/my_comm_test_mpi2
No protocol specified
Hello, I am 7 of 10. Sent 70 to Rank 0
Hello, I am 8 of 10. Sent 80 to Rank 0
Hello, I am 6 of 10. Sent 60 to Rank 0
Hello, I am 0 of 10. Received 10 from Rank 1
Hello, I am 0 of 10. Received 20 from Rank 2
Hello, I am 0 of 10. Received 30 from Rank 3
Hello, I am 1 of 10. Sent 10 to Rank 0
Hello, I am 2 of 10. Sent 20 to Rank 0
Hello, I am 3 of 10. Sent 30 to Rank 0
Hello, I am 5 of 10. Sent 50 to Rank 0
Hello, I am 0 of 10. Received 40 from Rank 4
Hello, I am 0 of 10. Received 50 from Rank 5
Hello, I am 0 of 10. Received 60 from Rank 6
Hello, I am 0 of 10. Received 70 from Rank 7
Hello, I am 0 of 10. Received 80 from Rank 8
Hello, I am 0 of 10. Received 90 from Rank 9
Hello, I am 4 of 10. Sent 40 to Rank 0
Hello, I am 9 of 10. Sent 90 to Rank 0
```

Required an explicit user-provided buffer. Without attaching a buffer, errors occured. Once the buffer was attached correctly, the program behaved similarly to the standard send, although the extra setup made it less convenient.

MPI_Rsend()
worked in the runs tested, producing the expected send and receive messages for both 4 and 16 processes. However, its behaviour is less reliable in general because it assumes that the matching receive has already been posted before the send occurs. In this program, that is not explicitly guaranteed, so even though it worked here, it may not always be safe or reproducible on different runs or systems.

MPI_Isend()
It is non-blocking, so it returns before the communication is necessarily complete. Without MPI_Wait(), the behaviour may be inconsistent. With MPI_Wait() added, it worked correctly, but the extra complexity was unnecessary for such a simple communication pattern. Tesed on 4,10 and 16 and worked fine on my system which is fine . This is likely because the program does very little actual work, so the overhead of communication dominates and any advantage of non-blocking sends is not noticeable.
Since the task is small and simple, the extra complexity of non-blocking communication is unnecessary. Based on reliability and reproducibility, MPI_Ssend() would be the better choice for this code, as it is simpler and gives more predictable behaviour without requiring additional buffer management or completion handling

Benchmark (Locally run from this point because server connection stopped running)

```
(base) dhaakirahfasetire@Dhaakirahs-Air HPQC % mpirun -np 4 ../bin/benchmark
Hello, I am 3 of 4. Sent 30 to Rank 0
Rank 3 took 0.000371 seconds to send to Rank 0
Hello, I am 2 of 4. Sent 20 to Rank 0
Rank 2 took 0.000381 seconds to send to Rank 0
Hello, I am 1 of 4. Sent 10 to Rank 0
Rank 1 took 0.000379 seconds to send to Rank 0
Hello, I am 0 of 4. Received 10 from Rank 1
Rank 0 took 0.000391 seconds to receive from Rank 1
Hello, I am 0 of 4. Received 20 from Rank 2
Rank 0 took 0.000000 seconds to receive from Rank 2
Hello, I am 0 of 4. Received 30 from Rank 3
Rank 0 took 0.000001 seconds to receive from Rank 3
```

The first receive is longer because rank 0 has to wait for the first message to arrive.After that, the other messages are often already sitting there waiting in MPI's buffer, so the next MPI_Recv calls complete almost instantly.

Part 2

Made use of pinpong /pinpong_arrag,c files
Plot.ipynb

The pingpong .c program measures the communication time between two process. The program initializes MPI and ensures that exactly two processes are used: a root process ,rank 0 and a client process ,rank 1. It takes a command-line argument specifying the number of ping-pong exchanges. A counter is initialized to zero, and timing begins using MPI_Wtime() the root process sends the counter to the client the client receives it increments it, and sends it back. The root receives the updated value. After the loop, the root process calculates the total elapsed time and the average time per ping-pong**,** which represents the communication latency.

```
(base) dhaakirahfasetire@Dhaakirahs-Air HPQC % mpirun -np 2 ../bin/pingpong 1000
Final counter   = 1000
Elapsed time    = 0.000760 seconds
Average time    = 7.600000e-07 seconds per ping-pong
```

Measuring latency

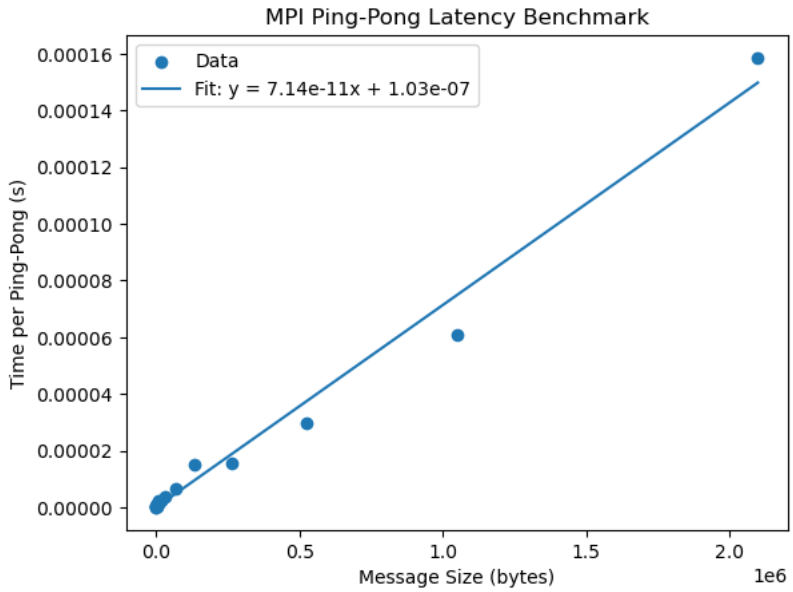
Number of Pings	Final Counter	Elapsed Time (s)	Average Time per Ping-Pong (s)
10	10	0.000236	2.36×10^{-5}
100	100	0.000424	4.24×10^{-6}
1000	1000	0.000512	5.12×10^{-7}
10000	10000	0.002835	2.84×10^{-7}
100000	100000	0.023560	2.36×10^{-7}

From the table above the results do converge . The program used 2 MPI processes while varying the number of ping-pong exchanges from 10 to 100000. The results show that for small numbers of pings, the average time per ping-pong is relatively large and inconsistent. As the number of pings increases, the average time decreases and stabilises then converges. This occurs because, for small iteration counts, timing overhead and system noise significantly affect the measurement. With larger numbers of pings, the communication cost dominates, resulting in a more accurate and stable estimate of latency.

Measure bandwith

The original code just sent one integer counter back and forth. The pinpong_array.c makes use of malloc() to create an array of a variable size then it send the entire array instead of a integer. The number of iterations are still counted using first element of the array while the rest of the array represents the data being transmitted. A second argument is message size (bytes) this calculated the number of elements using sizeof(int). The MPI send and recv calls are updated to send the full array rather than a single integer, allowing the communication cost to depend on message size.

To collect data for analysis, the program output was only the message size and the average time per ping-pong exchange. The results were saved to a file which was loaded to make the figure below.



Slope (m) = 7.135872947462853e-11 seconds per byte

Intercept (c) = 1.0324559997281861e-07 seconds

Bandwidth = 14013702421.026825 bytes/second

Bandwidth = 13364.508076693368 MB/s

The communication time was plotted as a function of message size and fitted with a linear model. The intercept in seconds represents the latency**,** this is the fixed cost associated with initiating communication between processes, independent of message size.The slope is the the time per byte required to transfer data. The bandwidth of the system is calculated as the reciprocal of the slope which is 1.40×10^10 byte/s which is approx.13GB/s. The results show a clear linear relationship between message size and communication time, indicating that the total communication cost consists of a fixed latency component and a variable component proportional to message size. This behaviour is consistent with theoretical expectations for MPI communication.

Part 3

Initial prediction, I think send and receive(Fullcopy) will be faster .

Code form last week was my_vector_serial _copy was editied to test the below .

Method	Size	Avg Runtime (s)
Fullcopy	1,000	0.000038
Bcast	1,000	0.000023
Scatter	1,000	0.000046
Fullcopy	100,000	0.000399
Bcast	100,000	0.008170
Scatter	100,000	0.009103
Fullcopy	1,000,000	0.003142
Bcast	1,000,000	0.006389
Scatter	1,000,000	0.009280

Each was run 3 times and average time was calculated for better benchmarking.
The results show that the full-copy implementation was consistently faster than both the broadcast and scatter approaches for all tested array sizes. This differs from the expected theoretical behaviour, where scatter is typically more efficient for large arrays due to reduced memory duplication and communication overhead. This can be explained aybe because the experiments were performed on a shared-memory system, where each MPI process runs on the same physical machine. In this case, memory allocation and initialisation are inexpensive while the over communications introduce overhead.

For small array sizes, all methods perform similarly due to the dominance of timing overhead. As the array size increases, communication costs become more significant, leading to a larger performance gap between the full-copy method and the communication-based approaches.

Step 2

Made use of manual/gather/reduce .c files

I predict reduce should be the fastest because it is best suited for this type of operation.Each process sends its local sum logically and the mpi does the summation which means the root gets the final total directly.

Method	Array Size	Average Runtime (s)
Manual send/recv	1,000	0.000099
Gather	1,000	0.000014
Reduce	1,000	0.000009
Manual send/recv	100,000	0.000059
Gather	100,000	0.000138
Reduce	100,000	0.000034
Manual send/recv	1,000,000	0.000420
Gather	1,000,000	0.000306
Reduce	1,000,000	0.000265

As predicted reduce has the fastest runtime.Gather is faster than send/recv when it is the smallest and then again for the largest but slower for the middle case because of one noticeably larger timing. Overall, the results support the prediction is most efficient, since it combines communication and summation into one optimized collective operation

Step 3

File name my_reduce_op…c
Comparing reduce.c file time

Comparriosn to reduce.c

Accuracy is good, gives the same answer as the reduce.c program

Reduce version

Array size = 1000

Processes = 4

Sum = 332834500

Runtime = 0.000077 seconds

Custom reduce version

Array size = 1000

Processes = 4

Sum = 332834500

Runtime = 0.000030 seconds

Times average table

Method	Array Size	Avg Runtime (s)
Custom Reduce	1,000	0.000008
Custom Reduce	100,000	0.000022
Custom Reduce	1,000,000	0.000020

Comparing the custom to the reduce the performance is similar but the suctomo is quicker sometimes.The runtime is almost constant with 100000 and 100000 being quite close. For the custom reduction, only one value per process is reduced so the communication cost is small and the differences are still within timing noise.